

Clase 138 — Desarrollo de exploits moderno

Parte: 5 — Explotación de sistemas y binarios · Fuente: *The Shellcoder's Handbook* · docs pwntools 📖

Duración estimada: 140 min · Nivel: Experto

Objetivo

Integrar todo lo aprendido en un flujo de **desarrollo de exploits moderno**: encadenar primitivas (info leak → cálculo de bases → escritura/control de flujo → ROP → shell), automatizar con **pwntools**, manejar la libc correcta y escribir exploits fiables y reutilizables. Verás cómo se combinan las técnicas para derrotar NX+ASLR+PIE+canary en un objetivo realista de laboratorio.

⚠ **Ética:** solo binarios propios o retos de CTF autorizados.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Diseñar** una cadena de explotación por primitivas (leak → base → control).
2. **Automatizar** el exploit con pwntools (ELF, ROP, libc, tubes).
3. **Resolver** la libc correcta y calcular bases dinámicamente.
4. **Encadenar** ROP/ret2libc con un info leak para vencer ASLR/PIE.
5. **Robustecer** el exploit (reintentos, badchars, entorno estable).

Temas

#	Tema	Por qué importa
1	Pensar por primitivas	Estructura mental del exploit
2	Info leak → base	Vencer ASLR/PIE
3	Cálculo de direcciones dinámico	Independencia de la versión
4	ROP + ret2libc combinados	Ejecución bajo NX
5	pwntools avanzado	Automatización real
6	libc-database / ret2dlresolve	Cuando no conoces la libc
7	Robustez y badchars	Que funcione siempre
8	Local → remoto	Retos en servidor

Definiciones y características

- **Primitiva de explotación:** capacidad atómica (leer, escribir, controlar flujo). *Clave:* los exploits se construyen encadenando primitivas.
- **Info leak:** filtra una dirección para derrotar ASLR/PIE. *Clave:* imprescindible en objetivos modernos.
- **Cálculo de base dinámico:** `base = leak - offset`. *Clave:* hace el exploit robusto entre corridas.
- **ret2dlresolve:** técnica que fuerza al enlazador dinámico a resolver un símbolo (p. ej. `system`) sin conocer libc. *Clave:* útil sin leak de libc.
- **pwntools tubes:** abstracción de I/O (`process`, `remote`, `recvuntil`, `sendlineafter`). *Clave:* el mismo exploit sirve local y remoto.
- **Robustez:** manejo de badchars, alineación, timing y reintentos. *Clave:* un exploit que solo funciona a veces no sirve.

Herramientas y preparación

```
pip install pwntools ROPgadget
# libc-database para identificar libc por un leak
git clone https://github.com/niklasb/libc-database
```

Laboratorio guiado

Entorno propio.

1. Reconocimiento del objetivo (binario de reto con NX+PIE+ASLR, sin canary o con canary a filtrar):

```
bash checksec ./target
```

2. Estructura el exploit en pwntools con una plantilla reutilizable:

```
python from pwn import * exe = context.binary = ELF("./target") libc = ELF("./libc.so.6") def
conn(): return remote(args.HOST, int(args.PORT)) if args.REMOTE else process(exe.path) io =
conn()
```

3. **Primitiva 1 — leak:** usa la vulnerabilidad para filtrar una dirección de libc (GOT/stack) y calcula la base:

```
python leak = u64(io.recvline().strip().ljust(8, b"\x00")) libc.address = leak -
libc.symbols["puts"] log.success(f"libc @ {hex(libc.address)}")
```

4. **Primitiva 2 — control de flujo:** construye una cadena ROP con la base ya conocida para `system("/bin/sh")` o un `one_gadget`, cuidando la alineación.

5. Si el binario es PIE, filtra también su base y usa `exe.address = code_leak - offset`.

6. Si no conoces la libc, identifícala por el leak con `libc-database` o aplica `ret2dlresolve` (`rop.ret2dlresolve(...)` en pwntools).

7. Robustece: envuelve en un bucle de reintentos, evita badchars y estabiliza el entorno (`env`, ASLR). Prueba `./exploit.py REMOTE HOST=... PORT=...` contra un `socat` local.

Ejercicios

1. Escribe una plantilla pwntools que alterne local/remoto con `args.REMOTE`.
2. Identifica una libc desconocida a partir de un leak con libc-database.
3. Añade filtrado de canary vía un leak previo.
4. Sustituye `system` por un `one_gadget` válido y verifica constraints.
5. Implementa `ret2dlresolve` en un binario sin leak de libc.
6. Añade lógica de reintento y mide la fiabilidad en 20 corridas.

Reto verificable

Desarrolla un exploit completo contra un binario de laboratorio con NX+ASLR (y PIE si te atreves) que obtenga shell filtrando la base de libc en runtime.

Criterio de aceptación: `./exploit.py` da shell interactiva de forma fiable ($\geq 8/10$ corridas) con ASLR activado, calculando la base dinámicamente.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Base de libc incorrecta	libc equivocada; identifícala con el leak
Funciona local, no remoto	Diferencias de libc/entorno; usa la del servidor
Crash intermitente	Badchars o alineación; revisa el payload
one_gadget no da shell	No se cumplen sus constraints; prueba otro
Leak desalineado	Falta <code>ljjust(8, \x00)</code> o offset del leak erróneo

Preguntas frecuentes

 **¿Siempre necesito un leak?** Con ASLR/PIE, casi siempre. Sin leak, técnicas como `ret2dlresolve` o `partial overwrite` pueden ayudar.

 **¿Cómo sé qué libc usa el reto?** Por un leak + libc-database, o porque el reto la proporciona.

 **¿Un exploit debe ser 100% fiable?** Apunta a la máxima fiabilidad; en CTF basta con que funcione consistentemente contra el servidor.

Referencias

- pwntools — <https://docs.pwntools.com/>
- libc-database — <https://github.com/niklasb/libc-database>
- `ret2dlresolve` (pwntools) — <https://docs.pwntools.com/en/stable/rop/ret2dlresolve.html>
- The Shellcoder's Handbook, 2e. Wiley.

Material descargable

-  [Guía en PDF](#) — versión imprimible de esta clase.

-  [Presentación \(PPTX\)](#) — deck para proyectar en clase.

Siguiente clase

[Clase 139 - Kernel exploitation: introduccion](#)